

República Bolivariana de Venezuela

Ministerio del Poder Popular para la Educación Universitaria

Universidad Politécnica Territorial

´´José Antonio Anzoátegui´´

Trayecto 2 Fase 2 PNF: Informática

Municipio Guanta, Edo. Anzoátegui

Informe

Profesor:

Lenis Hunmer

Alumno:

Johan Salazar

CI: 32.797.143

9-11-26

ÍNDICE

Introducción 3
DESARROLLO 4
Variable: 4
Tipo de dato 4
Enteros (int 4
Decimales/Flotantes (float / double): 4
Booleanos (bool): 4
2. El corazón de la POO (Cómo se organizan los programas) 4
Clase: 4
Objeto: 4
Atributos: 4
Métodos: 4
3. Los 4 Pilares de la Programación Orientada a Objetos 4
1. Abstracción: 5
2. Encapsulamiento: 5
Conclusión 6
Bibliografía 7

Introducción

En el ámbito del desarrollo de software actual, la capacidad de estructurar la información y modelar problemas del mundo real de forma eficiente es fundamental. La evolución de la programación ha transitado desde la ejecución secuencial de instrucciones hasta paradigmas más avanzados como la Programación Orientada a Objetos (POO), la cual permite gestionar la complejidad mediante la modularidad y la reutilización de código.

El presente documento aborda los conceptos fundacionales que todo estudiante de informática o desarrollador debe dominar. Se parte desde los elementos más elementales —como el almacenamiento de datos en variables y sus tipos— hasta llegar a las estructuras lógicas complejas de la POO, examinando el concepto de clase, objeto y los cuatro pilares esenciales (abstracción, encapsulamiento, herencia y polimorfismo) que sustentan el diseño de software moderno y escalable.La Programación Orientada a Objetos es un paradigma que replantea la estructura del código al imitar entidades del mundo real, agrupando los datos (atributos) y las acciones que los manipulan (métodos) en un mismo contenedor lógico

DESARROLLO

Los elementos básicos (Los datos)

Antes de construir estructuras complejas, necesitamos saber cómo guardar y representar la información.

Variable: Es una "caja" en la memoria de la computadora que almacena un valor. Puede cambiar ("variar") a lo largo del programa (por ejemplo, la puntuación de un juego o el nombre de un usuario).

Tipo de dato: Define qué tipo de información puede guardar una variable. Los más comunes son:

Enteros (int): Números sin decimales (ej. 25).

Decimales/Flotantes (float / double): Números con decimales (ej. 3.14).

Cadenas de texto (string): Letras o palabras (ej. "Hola mundo").

Booleanos (bool): Valores de verdadero o falso (true / false).

2. El corazón de la POO (Cómo se organizan los programas)

La Programación Orientada a Objetos imita al mundo real: agrupamos los datos y las acciones que los manipulan en "bloques" lógicos.

Clase: Es el molde, plano o plantilla. Define cómo serán las cosas (por ejemplo, una clase Perro que dice que todos los perros tienen raza y edad, y pueden ladrar).

Objeto: Es la instancia (la materialización) de una clase. Si la clase es el plano de un coche, el objeto es el coche real que puedes manejar en la calle.

Atributos: Son las características o variables que pertenecen a una clase (ej. el color de un perro, su peso, su nombre).

Métodos: Son las acciones o funciones que puede realizar un objeto (ej. correr(), comer(), ladrar()).

3. Los 4 Pilares de la Programación Orientada a Objetos

Esta es la "teoría pesada" que todo programador aprende para diseñar sistemas limpios, reutilizables y seguros:

1. Abstracción: Consiste en aislar los elementos esenciales de un objeto ignorando los detalles complejos que no importan. (Ejemplo: Para conducir un coche usas el volante y los pedales; no necesitas saber cómo funciona la inyección de combustible a nivel de ingeniería).

2. Encapsulamiento: Es el principio de ocultar los datos internos de un objeto para protegerlos. Usamos "candados" (llamados modificadores de acceso como private o public) para que nadie pueda modificar información delicada por accidente.

3. Herencia: Permite que una clase herede características y métodos de otra clase. Esto evita repetir código. (Ejemplo: Tienes una clase padre Animal con atributos como edad, y clases hijas como Perro y Gato que heredan esos atributos y añaden los suyos propios).

4. Polimorfismo: Significa "muchas formas". Permite que un mismo método o acción se comporte de manera diferente según el objeto que lo esté ejecutando. (Ejemplo: Si tienes un método hacerSonido(), un objeto Perro ladrará, pero un objeto Gato maullará usando exactamente el mismo nombre de método).

Conclusión

La Programación Orientada a Objetos (POO) representa un cambio fundamental en la forma en que pensamos y resolvemos problemas a través del código. Mientras que los elementos básicos como las variables y los tipos de datos son los bloques de construcción elementales (el equivalente a las letras y palabras en un idioma), las clases y los objetos nos permiten estructurar sistemas complejos imitando al mundo real.

Dominar los cuatro pilares (abstracción, encapsulamiento, herencia y polimorfismo) no es solo aprender sintaxis de un lenguaje, sino adoptar una mentalidad de diseño. Esto se traduce en software más limpio, modular, seguro y fácil de mantener a lo largo del tiempo, facilitando el trabajo en equipo y la escalabilidad de los proyectos tecnológicos.

Bibliografía

1. Booch, G. (2006). Object-Oriented Analysis and Design with Applications. Addison-Wesley Professional.

(Obra de referencia fundamental sobre los principios teóricos y el diseño modelado mediante objetos).

2. Deitel, H. M., & Deitel, P. J. (2017). Java: Cómo programar (10ª ed.). Pearson Educación.

(Excelente manual académico para comprender desde los tipos de datos y variables hasta los pilares de la POO con ejemplos prácticos).

3. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

(El libro clásico de la "Gang of Four" sobre buenas prácticas y reutilización de código orientado a objetos).

4. Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall.

(Guía esencial sobre cómo estructurar clases, métodos y variables de forma limpia y mantenible).

5. Schildt, H. (2018). Java: The Complete Reference (11ª ed.). McGraw-Hill Education.

(Texto técnico detallado sobre la implementación de clases, herencia, polimorfismo y encapsulamiento en lenguajes modernos).